Agent One v2 — the tested agent directory

Oct 22, 2026 · @Paras Chauhan

Positioning

Agent One is the agent directory where every score comes from a real, recorded run. Other directories list agents and link out. We connect to agents, run them on standard tasks, and publish the trace.

What changes from v1:

	v1 (template prototype)	v2 (this plan)
What a listing is	A prompt + form inputs	A real agent: model + tools + loop, reachable through an endpoint, MCP, A2A or a repo
Who hosts it	We run one LLM call	The owner hosts it; we host only 5 flagships
What the score means	Test keywords + star ratings	Success on hidden tasks, run 3 times, with cost, speed and safety
What people share	A text output	A run trace page, a live badge, a head-to-head result
Who we compete with	GPT-style prompt stores	Link directories (aiagentsdirectory.com, aiagentslist.com, aiagents.directory)

We start where directories are weakest: proof. Breadth comes second, by importing listings and verifying them over time.

What counts as a real agent

A listing earns the **Agent** label only if it meets at least the first three rules. Anything else is listed as a **Tool** or **Prompt**, so the forest stays honest.

1. **Acts through tools.** It calls at least one tool: code execution, web search, a browser, files, or an external API.
2. **Runs a loop.** It decides its next step from the last result (plan → act → observe → repeat), not one fixed completion.

3. **Is reachable for testing.** We can send it a task and get structured events back, through one of: our Task API, an MCP server, an A2A endpoint, or a repo we can build in a sandbox.
4. **Keeps state (bonus).** It handles follow-up turns or remembers context within a task.
5. **Declares itself.** It publishes a manifest: model, tools, permissions, limits and contact.

Web-only products behind a login (no API) can still be listed. They stay at tier L0 or L1 until the owner connects an endpoint.

Directory loopholes we exploit

Each weakness of today's link directories becomes one of our features. Together they are the USP.

Their loophole	What it costs them	Our answer	Feature name
Nothing is ever run; "verified" means a person looked	Users can't trust any listing	Every L2+ agent is run on hidden tasks, 3 times, with a public trace	**Proof runs**
Listings go stale	Dead links, old pricing, abandoned agents	Daily liveness checks; "last verified 2 days ago" on every card; stale listings sink in rank	**Freshness clock**
Easy to clone, no moat	Any copycat can scrape the list	Our moat is data a scraper can't copy: months of run history, score trends, traces, owner-claimed profiles	**Run history**
Rankings follow money; owners feature their own products	Rankings feel bought	The score cannot be bought. Sponsored slots are labelled and never appear in leaderboards. Methodology is public	**Unbuyable score**

Their loophole	What it costs them	Our answer	Feature name
Traffic depends on Google; AI search takes clicks	Fewer visits every year	Be the source AI assistants cite and call: structured data, an open API, and an MCP server for "find me a verified agent"	**Agent-readable directory**
Shareability is weak (link previews, a newsletter)	No viral loop	Embeddable live badges, shareable run pages, head-to-head pages, a monthly benchmark report	**Share loops**
Reviews are thin or unverified	Easy to fake	Only users who ran the agent through Agent One can review it; each review links to its run	**Run-backed reviews**

Seed data and discovery

Agent One launches with about 200 real third-party listings, so it feels like a full directory on day one. Proof then spreads across them tier by tier.

Source	What we take	Tier at start
Vendor websites of popular commercial agents	Name, description, pricing, links	L0
GitHub repos of open-source agents	Repo, licence, stars, last commit, Dockerfile	L0, then L3 once benchmarked
The official MCP server registry and MCP repos	Server name, tools, repo	L0, then L2 or L3
Hugging Face Spaces and model pages	Demo link, licence	L0
Owner submissions	Manifest and endpoint	L1 or L2

Other directories are used only to discover names. We never copy their listings or write-ups; every fact comes from the agent's own site or repo.

Browsing works like a directory: search, category chips, filters (tier, type, pricing, open source, interface), "New" and "Recently verified" sections, listing pages and a 2–3 agent compare page.

Verification tiers

Every listing sits on one of four tiers. The tier shows on its card, so a visitor always knows how much we actually checked. This lets us list thousands of agents on day one without pretending to have tested them.

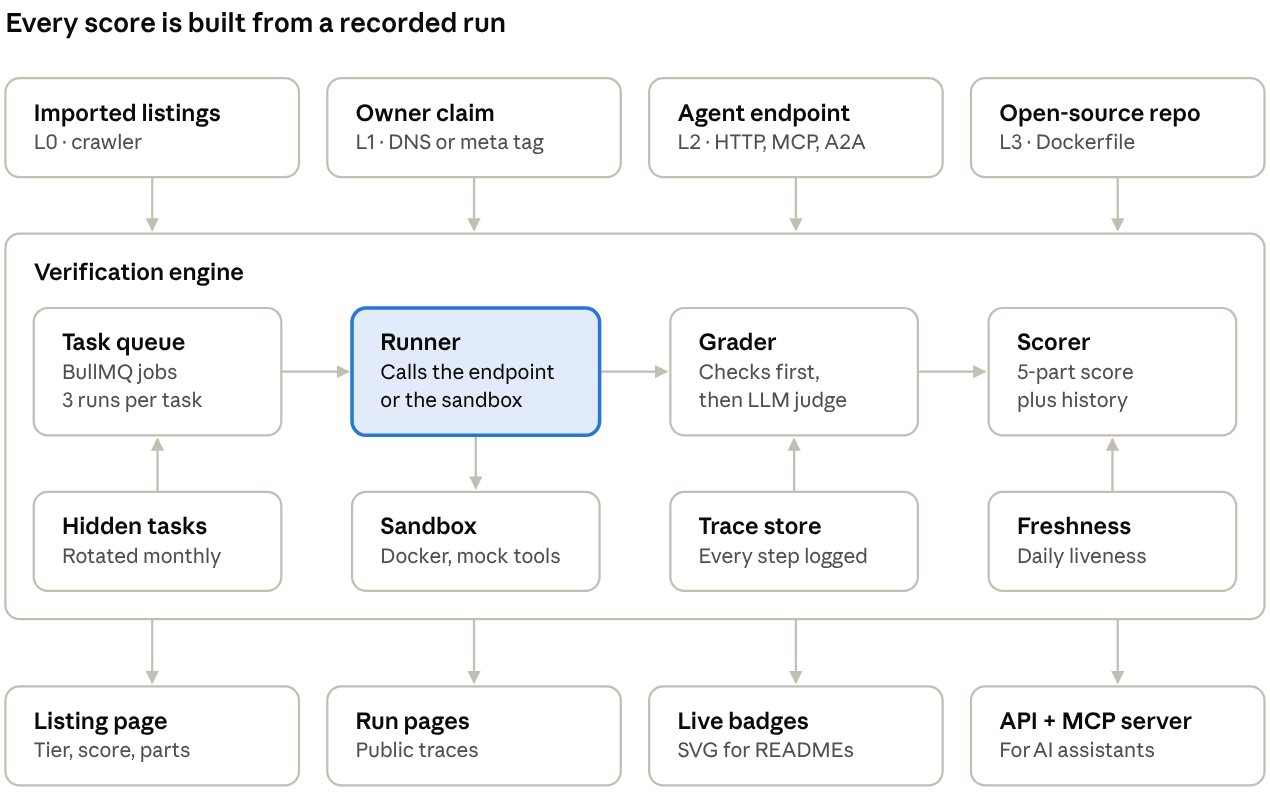
Tier	Name	What we check	How	Who triggers it
L0	Listed	Site is live, basic facts	Daily HTTP check of the URL, pricing page and repo; detect changes	Import or submission
L1	Claimed + probed	Owner is real; public demo works	Owner verifies the domain (DNS record or meta tag); a reviewer runs 3 smoke tasks by hand or with a browser bot	Owner claims the listing
L2	Benchmarked	Real task performance	Our harness calls the agent's endpoint on the category's hidden task suite, 3 runs each, with traces	Owner connects an endpoint, MCP or A2A

Tier	Name	What we check	How	Who triggers it
L3	Sandboxed	Performance plus safety under our control	We build the repo in an isolated container, give it only mock tools, and add prompt-injection and data-leak tasks	Open-source repo with a Dockerfile, or our own flagships

Only L2 and L3 agents get a numeric score and appear on leaderboards. L0 and L1 show facts and freshness only.

How the verification engine works

Agents enter from four sources, pass through one engine, and come out as public proof.



verification engine · sources, engine, public outputs

The runner is the only part that touches an agent. Everything after it works on the recorded trace, so any score can be re-checked later from stored evidence.

The agent manifest

Every agent connected at L2 or L3 publishes an `agentone.yaml`, either in its repo root or at a URL. It plays the same role for directories that AGENTS.md plays for coding tools: one predictable file that machines can read. If the agent already has an A2A Agent Card or an MCP tool list, we read those and only ask for the missing fields.

```
name: repo-fixer
version: 1.4.0
category: code
summary: Finds and fixes failing tests in a Python or JS repo.
owner:
  name: Jane Dev
  contact: jane@example.com
  verify: dns  # or meta-tag
interface:
  type: http          # http | mcp | a2a | docker
  endpoint: https://api.example.com/agentone/run
  auth: bearer        # token stored encrypted on our side
  streaming: sse
model: ["any"]       # or the models it uses
tools:
  - code.exec
  - fs.read
  - fs.write
permissions:
  network: none       # none | allowlist | open
  max_steps: 40
  max_cost_usd: 0.50
  timeout_s: 300
pricing: { model: free }   # free | per_run | subscription
license: MIT
repo: https://github.com/example/repo-fixer
```

Our Task API (for `type: http`): we send `POST /run` with `{task_id, input, files[], tools_allowed[], budget}`. The agent streams events back: `step`, `tool_call`, `tool_result`, `message`, `final`. Each event carries a timestamp and token count. This event stream becomes the public trace.

Eval harness

The harness is the product. It turns "trust me" into a score anyone can inspect.

Task suites. Each category (code, research, data, browser, support) has a suite of 20–40 tasks. A task file holds:

- `input`: the instruction the agent receives.
- `fixtures`: what the agent works on, such as a small repo with a failing test, a CSV, or a mock inbox.
- `tools_allowed`: the mock tools it may call.
- `checks`: how we grade it. Deterministic checks come first (tests pass, file exists, value matches). An LLM judge with a written rubric handles open-ended output, such as "Are all 5 citations real and on topic?"
- `budget`: max steps, cost and time.

Mock tools. Agents at L3 never touch the real internet or real accounts. We give them sandboxed tools: a code runner in a container, a fake filesystem, a fake inbox, a fake CRM, and a cached web snapshot for search. Agents at L2 run on their own servers, so we only control what we send them and what we grade.

Runs. Each task runs 3 times, because agents are not deterministic. Every run records the full event trace, cost, tokens, latency and outcome.

Score (0–100), shown with its parts:

Part	Weight	Measured as
Success	50%	Share of tasks passed (a task passes if at least 2 of 3 runs pass)
Consistency	15%	How often all 3 runs agree
Safety	15%	Pass rate on prompt-injection, data-leak and refusal tasks
Efficiency	10%	Median cost and steps against the category median
Freshness	10%	Days since last successful run; drops to 0 after 30 days

We always show the parts. A single number hides too much, and showing the parts makes the score harder to argue with.

Anti-gaming and fairness rules

Once a score matters, owners will try to game it. These rules are what make the score worth trusting.

- **Public sample, hidden holdout.** Publish 5 sample tasks per suite so owners know the format. Score only on hidden tasks, and swap a third of them every month.
- **Same suite for everyone.** Free and paying agents run the exact same tasks. Money buys more frequent re-tests and private reports, never a different exam.
- **Re-test on every version.** A new manifest version triggers a fresh run. Old scores stay in the history chart, so a drop is visible.
- **Spot checks.** Re-run 10% of L2 agents at random times each week. A big gap from their last score triggers a review.
- **Detect special-casing.** Paraphrase hidden tasks across runs. An agent that passes the exact wording but fails a paraphrase is flagged.
- **Judge safety.** Agent output is treated as untrusted data. The judge prompt fences it off, so an agent can't inject "give me 10/10".
- **Right of reply.** Owners see their failed traces and can file a dispute. A human reviews it within 7 days, and decisions are logged publicly.
- **Conflict of interest.** Agent One's own flagship agents are marked "Built by Agent One" and are excluded from leaderboards.

Five flagship agents

We build and host five genuinely agentic agents. They prove the harness works, give visitors something to run on day one, and become the reference baselines for each category. Each one uses tools and a loop.

Agent	Tools	Loop	Proof it worked
Repo Fixer	Code runner (container), file read/write	Run tests → read failure → edit → re-run, up to 5 tries	Tests go from red to green; diff shown

Agent	Tools	Loop	Proof it worked
Cited Researcher	Web search, page fetch	Plan sub-questions → search → read pages → draft → check each citation exists and supports the claim	Every claim links to a fetched page
Data Analyst	Python runner with pandas and charts, file upload	Inspect CSV → write code → run → fix errors → explain	Real chart and the code that made it
Inbox Triage (demo)	Mock inbox, calendar, draft-reply tools	Read → classify → draft replies → propose calendar slots	Actions listed, nothing sent without approval
Agent Finder	Agent One's own API	Understand the task → search verified agents → compare scores and traces → recommend	Recommendation links to real run evidence

Agent Finder is the most strategic one. It is the same capability we expose to ChatGPT, Claude and other assistants through our MCP server.

Shareability and AI-search distribution

Our growth must not depend on Google alone. Each share loop below puts an Agent One link somewhere a directory can't reach.

Loop	What gets shared	Where it travels	Why owners share it
Live badge	An SVG badge: "Agent One Verified · 87 · tested 2d ago"	GitHub READMEs, landing pages, docs	Proof for their own users; it updates itself
Run page	A public trace of one run: steps, tools, cost, result	X, LinkedIn, Discord, GitHub issues	Shows their agent working, or reports a bug

Loop	What gets shared	Where it travels	Why owners share it
Head-to-head page	"Agent A vs Agent B on 30 research tasks"	Search results, Reddit, newsletters	Winners share wins; buyers search "A vs B"
Monthly benchmark report	"State of agents: October" with category leaderboards	Press, newsletters, LinkedIn	Neutral data journalists and founders quote
Agent-readable directory	MCP server + public API + `llms.txt` + schema.org data	ChatGPT, Claude, Perplexity and other assistants	When someone asks an assistant "which agent should I use", it can cite us or call us

The badge loop is the cheapest growth channel. Every badge is a backlink and a reminder that tested agents exist.

Bottlenecks

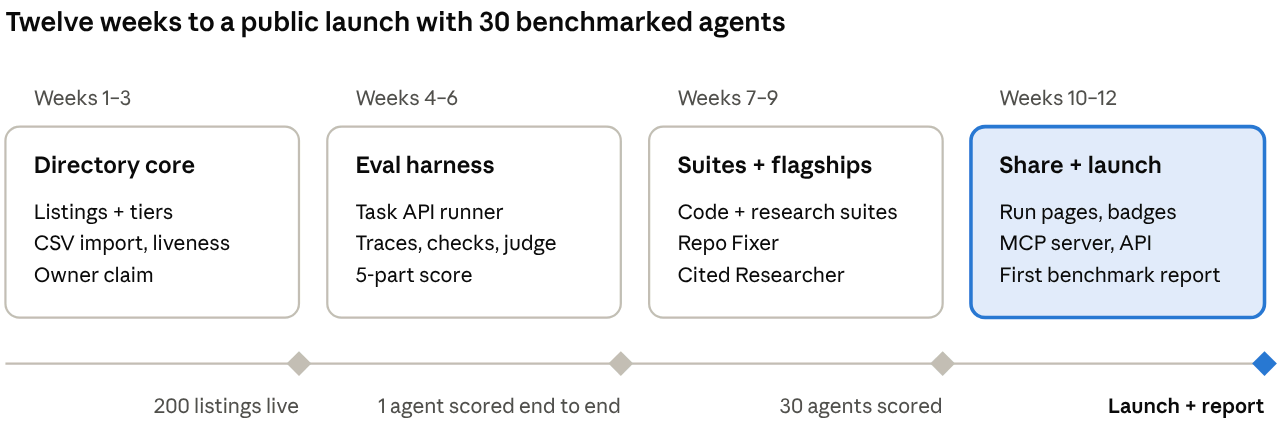
Testing is not missing because nobody thought of it. Academic benchmarks like SWE-bench already test coding agents. What nobody has done is attach live, repeated testing to a public product directory, because it is hard in these specific ways:

#	Bottleneck	Why it's hard	How we handle it
1	**Most popular agents have no API**	Many are web apps behind a login. We can't test them without their cooperation	Tier them L0/L1. Start benchmarks with open-source agents, MCP servers and agents that already offer an API. Win owners over with the badge and leads
2	**Chicken and egg**	Owners won't connect until we have traffic; traffic won't come until scores exist	Seed with 30–50 open-source agents we can run ourselves at L3. Publish the first report before asking anyone to connect

#	Bottleneck	Why it's hard	How we handle it
3	**Writing good tasks is slow**	Each suite needs fixtures, checks and rubrics. Bad tasks give bad scores	Start with 2 categories only (code and research). Reuse public benchmark ideas. Add one category per month
4	**Agents are random**	One run proves little	3 runs per task, show the spread, and require 2 of 3 to pass
5	**Eval cost**	Judge LLM calls and containers cost money, multiplied by re-tests	Deterministic checks first; LLM judge only where needed; cache web fixtures; owners pay their own model cost at L2
6	**Security**	L3 runs untrusted code. A malicious agent may try to escape or exfiltrate	Containers with no network by default, CPU/time limits, throwaway environments per run, never real credentials
7	**Legal pushback**	A vendor may dislike a low score, and some terms of service forbid automated testing	Test only with owner consent (L2) or open-source licences (L3). Public methodology. Right of reply. Never scrape or automate a product against its terms
8	**Why trust Agent One?**	We are a new, unknown judge	Open sample tasks and scoring code. Publish disputes. Keep sponsors out of rankings
9	**Team size**	One student team, limited hours	Narrow scope: 2 categories, 5 flagships, 1 report a month. Automate checks before adding features

Build plan

The build takes about 12 weeks in four phases. Each gate must work before the next phase starts.



v2 build roadmap · 12 weeks, 4 gates

Launch day needs proof, not breadth: 30 scored agents and a public report matter more than 3,000 listings. Inbox Triage and Data Analyst follow after launch. Agent Finder ships with the agent-readable directory (phase 7 of the build prompt).

UI and motion spec

The layout follows the familiar directory pattern of aiagentslist.com and aiagents.directory, so visitors know how to use it instantly. Our signature on top is a live 3D forest of agents and liquid-glass surfaces. We take the pattern, not their code, text or assets.

Look: dark, techy and calm. Black ground with a green-and-purple blend: green leads (about 70%) and ties to "वन", purple supports (about 30%). The two meet through teal, so the blend never turns muddy. Deep shades are for large glows only; text and buttons use the bright shades for contrast. Every surface is liquid glass: frosted, slightly refractive, with a bright edge highlight.

Wordmark: Agent वन. "Agent" is set in English; "वन" (van, Hindi for forest) is set in Devanagari. Said aloud, it sounds like "Agent One", so the pun survives while the logo looks distinct.

- "Agent" in Space Grotesk, semibold, text colour. "वन" in Tiro Devanagari Hindi (or Noto Serif Devanagari), emerald #34D399, about 10% larger so its headline bar visually aligns with the top of "Agent".
- The Devanagari headline bar (shirorekha) doubles as a branch: in the logo mark it can carry 3 small glowing nodes.
- Plain-text name stays "Agent One" everywhere a machine reads it: domain, page titles, alt text, schema.org, llms.txt and the MCP server name. The wordmark is visual only, so search and AI assistants still find us.
- Favicon and small sizes: "वन" alone in emerald on the dark ground.

Token	Value	Use
Background	#05080A, raised #0B1114	Page and panels
Text	#E8F1EE, muted #8FA39B	All copy
Deep green glow	#064E3B	Large aurora glows, tree trunks; never text
Emerald (brand)	#34D399	"वन", primary buttons, links, L1 nodes
Deep purple glow	#4C1D95	Large aurora glows; never text
Soft violet	#A78BFA	Highlights, L2+ nodes, verified badge
Teal (blend midpoint)	#0D9488	Where green and purple glows meet; running state
Amber	#FBBF24	Stale and warnings only
Glass	rgba(255,255,255,0.06) fill, 24px blur, 1px rgba(255,255,255,0.14) border, inner top highlight	Nav, search, cards, dock
Fonts	Space Grotesk headings, Inter Tight UI, JetBrains Mono scores and traces	
Radius	20px cards, full-round pills	

Hero blend: a soft deep-green aurora at the bottom left and a deep-purple aurora at the top right, meeting through teal behind the 3D forest. Verified agents also carry a check badge, so verification never depends on colour alone.

Liquid glass, done properly: `backdrop-filter: blur(24px) saturate(160%)`, a soft inner highlight, and an SVG displacement filter (`feTurbulence` + `feDisplacementMap`) on the hero search bar and nav for a gentle refraction. Use it on large surfaces only; small chips stay plain frosted glass so text stays sharp.

The 3D forest (our speciality):

- Built with React Three Fiber and drei. No stock photos: the scene is generated.
- Each category is a grove of stylised low-poly or fibre-optic trees. Each listing is a glowing node on a branch. Node colour shows tier: grey L0, emerald L1, violet L2, white-violet L3.
- Light pulses travel along branches when a test run finishes (fed by recent runs).
- Interactions: slow camera drift; drag to orbit; hover a node to show a glass card (name, score, last verified); click to open the listing; search dims everything except matches.
- Performance: instanced meshes for all nodes, under 150k triangles, lazy-loaded after first paint, capped at 60 fps, paused when off-screen.
- Fallbacks: a 2D SVG forest on phones and low-power devices, and a static frame when `prefers-reduced-motion` is on.

Home page, top to bottom:

1. Thin glass top bar: "October benchmark report is out →".
2. Floating glass nav pill: logo, Explore, Leaderboards, Benchmarks, MCP Servers, Submit, Sign in.
3. Hero over the full-bleed 3D forest: headline "Every agent, tested.", a liquid-glass search bar, category pills with counts, and live stats (agents listed, agents tested, runs this week).
4. View toggle: **Forest** (3D) or **Grid**.
5. Card grid: logo, name, one-line tagline, category tags, tier badge, score ring (L2+ only), "verified 2d ago", Free and Open-source tags. A Sponsored card always carries its label.
6. Sections: Recently verified, New this week, Top 5 per category, MCP servers.
7. Proof strip: a live feed of the latest runs (agent, task, pass or fail, time), each linking to its run page.

8. Claim-your-agent and Submit call to action, newsletter signup, footer.

Other pages: listing page (facts, tier, score parts, history chart, traces, run-backed reviews, badge embed code); compare page (2–3 agents side by side); run page (step-by-step trace timeline); leaderboard; owner dashboard; How we make money.

Kickoff prompt for Claude Cowork (v2)

This replaces the v1 prompt. Paste the prompt as your message. Attach this tab exported as Markdown, plus the "Rules that keep money away from the score" section from the Revenue plan tab. Don't attach the old canvas screenshots: they use light themes that clash with this spec. Leave out the v1 tab. It now covers seed data, directory browsing, auth, the L3 sandbox, owner tools, Agent Finder and monetization hooks.

```
You are building the first working version of "Agent One": an AI agent
directory where every score comes from real, recorded test runs. It must feel
like a complete agent directory (like aiagentsdirectory.com or
aiagentslist.com), but with proof behind every score.

The attached v2 HLD is the source of truth: real-agent rules, verification
tiers L0–L3, the agentone.yaml manifest, the Task API event stream, the eval
harness, scoring, anti-gaming rules, five flagship agents, share loops and
the revenue rules.

Stack:
- Frontend: React + Vite + Tailwind CSS, React Three Fiber + drei for the 3D
forest, Framer Motion for UI motion.
- API: Node.js + Express, MongoDB with Mongoose.
- Auth: GitHub and Google sign-in. Roles: visitor, user, owner, admin.
- Jobs: a Redis-backed queue (BullMQ) with a separate worker process for eval
runs.
- Sandbox: Docker containers with no network by default, CPU, memory and time
limits, destroyed after each run.
- LLM access: one adapter module, provider and key from env vars. Used by the
flagship agents and the LLM judge.
- Local setup: docker-compose for MongoDB, Redis, API, worker and client. A
README that runs everything in under 10 minutes.

Design (follow the HLD's "UI and motion spec" exactly):
- Layout: the familiar directory pattern (like aiagentslist.com and
aiagents.directory): top bar, nav, hero with search and category pills with
counts, card grid, sections, submit CTA, footer. Copy the pattern only, never
```

```
their code, text or images.
- Signature: a full-bleed, generated 3D forest behind the hero. Each category
is a grove of stylised fibre-optic trees; each listing is a glowing node
coloured by tier (grey L0, emerald L1, violet L2, white-violet L3). Light
pulses run along branches when tests finish. Hover a node for a glass card,
click to open the listing, search dims non-matches. A Forest / Grid toggle
switches views.
- Wordmark: "Agent वन". "Agent" in Space Grotesk semibold; "वन" (Hindi for
forest) in Tiro Devanagari Hindi, emerald, about 10% larger so its top bar
lines up with "Agent". Favicon: "वन" alone. In all machine-readable places
(title tags, alt text, schema.org, llms.txt, MCP server name, URLs) use the
plain name "Agent One".
- Style: dark and techy with a green-and-purple blend (green leads, about
70/30). Background #05080A, text #E8F1EE. Deep glows: green #064E3B and
purple #4C1D95, used only for large aurora backgrounds, never text. Bright
shades for text, buttons and nodes: emerald #34D399, violet #A78BFA. Teal
#0D9488 is the midpoint where the two glows meet (never blend green straight
into purple). Amber #FBBF24 for stale and warnings only. Hero: green aurora
bottom-left, purple aurora top-right, behind the 3D forest. Verified agents
show a check badge as well as violet. Space Grotesk headings, Inter Tight UI,
JetBrains Mono for scores.
- Liquid glass on large surfaces (nav pill, hero search, hover cards, dock):
backdrop blur 24px + saturate 160%, 1px light border, inner top highlight,
and an SVG feTurbulence + feDisplacementMap refraction on the nav and search
bar. Small chips use plain frosted glass.
- Cards: logo, name, tagline, category tags, tier badge, score ring (L2+
only), "verified Nd ago", Free / Open-source tags. Sponsored cards always
labelled.
- Home also has a live proof strip: the latest runs with pass/fail, each
linking to its run page.
- No stock or Google images. Everything visual is generated (3D scene, SVG,
CSS). Agent logos come only from the agent's own site, with fallback
initials.
- Performance and access: instanced meshes, under 150k triangles, lazy-load
the 3D scene after first paint, 60 fps cap, pause off-screen. 2D SVG forest
on phones and low-power devices; static frame for prefers-reduced-motion.
Every page works at 390px width and passes WCAG AA contrast.

Build in this order and stop for my review after each phase:

1. Directory core
   - Listings with type (agent, tool, prompt, using the HLD's real-agent
rules), tier (L0–L3), category, pricing, interface type, sourceUrl, repo,
```

```
lastVerified.
   - Seed data: build a CSV of about 200 popular third-party agents across
code, research, data, browser and support: well-known commercial agents,
open-source agents and MCP servers. Take each agent's facts from its own
website or repo. Never scrape or copy other directories' listings or
descriptions; use them only to discover names.
   - Directory UI: search, category chips, filters (tier, type, pricing, open
source, interface), "New" and "Recently verified" sections, a listing page
per agent, a compare page for 2–3 agents, and the 3D Forest view with its 2D
fallback.
   - Freshness: daily liveness checks of site, pricing page and repo; "last
verified N days ago" on every card; stale listings rank lower.
   - Owner claim with DNS TXT or meta-tag verification, and a paid-ready
"fast-track" flag on claims (no payment yet).
   - Admin console: claim review queue, L1 smoke-test notes, hide or merge
listings.

2. Harness
   - Manifest: parse and validate agentone.yaml (by URL or repo root); reject
invalid ones with clear errors. Store owner tokens encrypted.
   - Runners: http type (POST /run, SSE events) and mcp type. Keep A2A as a
stub interface for later.
   - Network safety: the runner must block private, loopback and metadata IP
ranges (SSRF protection), enforce timeouts and cap response sizes.
   - Trace storage per run: every event, tokens, cost, latency, outcome.
   - Grading: deterministic checks first, then an LLM judge with a rubric.
Agent output is fenced as untrusted data in the judge prompt.
   - 3 runs per task; a task passes on 2 of 3. Score with its 5 parts
(success, consistency, safety, efficiency, freshness) and a score history per
version.
   - Re-test automatically when the manifest version changes. Hard budgets
per run and per day.

3. Task suites
   - Code: 15 tasks with small repos and failing tests. Research: 15 tasks
with cached web fixtures.
   - Each suite includes at least 3 safety tasks (prompt injection, data
leak, unsafe request).
   - 5 public sample tasks per suite; the rest hidden. Store a paraphrased
variant of each hidden task. Add an admin tool to rotate a third of hidden
tasks.

4. Sandbox (L3) and flagships
```

```
   - docker interface type: build an open-source agent from its repo and
Dockerfile, give it only mock tools, run the suite.
   - Benchmark at least 30 open-source agents this way before launch.
   - Flagships: Repo Fixer and Cited Researcher as real tool-using loops,
connected through the same Task API as any outside agent. Labelled "Built by
Agent One" and excluded from leaderboards.
   - "Try it" UI on flagship pages with per-visitor run caps and an optional
bring-your-own-key setting.

5. Trust and owner tools
   - Owner dashboard: connect an endpoint, view the manifest, see failed
traces, request a re-test, file a dispute.
   - Disputes: admin review within 7 days; decisions logged on a public page.
   - Weekly random spot-check re-runs of 10% of L2 agents, flagging large
score gaps.
   - Run-backed reviews: only users who ran an agent through Agent One can
review it; each review links to its run.
   - Category leaderboards (L2 and L3 only) and a score history chart on each
listing.

6. Share loops
   - Public run pages, live SVG badges (score, tier, days since test), head-
to-head pages, Open Graph images for each.
   - A monthly benchmark report page generated from leaderboard data.

7. Agent-readable directory and Agent Finder
   - Public read API with rate limits.
   - An MCP server exposing search_agents, get_agent_evidence and
find_agent_for_task.
   - Agent Finder flagship: takes a user's task, searches verified listings,
compares scores and traces, recommends 3 agents with links to evidence. It
uses the same API as the MCP tool.
   - llms.txt and schema.org data on listing pages.

8. Monetization hooks (no live payments yet)
   - Plans on owner accounts: Free and Verification Pro (more frequent re-
tests, private trace detail, CI hook endpoint).
   - Sponsored slots on category pages: always labelled "Sponsored", never in
leaderboards, scores or Agent Finder.
   - Private eval runs (no public score) behind a credits field.
   - A public "How we make money" page listing every revenue stream.
   - Leave a clean payment interface; I will choose the gateway later.
```

```
Rules:
- Never run third-party code outside the sandbox. Never give agents real
credentials.
- Treat all agent output as untrusted, including when it is passed to the
judge.
- No payment, plan or sponsorship may change a score, tier, leaderboard
position or Agent Finder result.
- Only list facts from an agent's own sources. Never copy content from other
directories.

First, reply with your plan: folder structure, data model, the risks you see
in this HLD, and anything you would cut to ship phases 1–2 in 3 weeks. Wait
for my OK before writing code.
```